SYMMETRY-RESTRICTED SAT ENCODINGS, LEAN 4 FORMALIZATION, AND VERIFICATION LIMITS FOR CONWAY'S 99-GRAPH

TECHNICAL NOTE / BENCHMARK REPORT

Antonio Machuca
Department of Computer Science and Statistics
Universidad Rey Juan Carlos
Madrid, Spain
am.machuca.2023@alumnos.urjc.es
Permanent contact: contactoantoniomachuca@gmail.com

October 3, 2026

ABSTRACT

Conway's 99-graph problem asks whether a strongly regular graph with parameters $(99, 14, 1, 2)$ exists. Cited results restrict the prime divisors of the automorphism group order to $\{2, 3\}$. This note records a Lean 4 library, SAT compilers, and solver logs, and it uses a four-state taxonomy—PROVED, COMPILED, EXPLORED, and PENDING—to separate kernel-checked theorems, checker-certified formula refutations, tested compilers, and open claims. A SAT claim is PROVED only when the cited CNF is the file that was checked, its solver log ends in `s UNSATISFIABLE`, and `drat-trim` reports `s VERIFIED` on that pair. In Lean 4, we kernel-verify core structural invariants: K_4-freeness, 1-factor neighborhood decomposition $7K_2$, diameter at most 2, and decidability of the adjacency matrix equation. These proofs depend solely on the foundational axioms `[propext, Quot.sound]`. We also formalize the arithmetic contradictions that underlie character-theoretic obstructions. This distinguishes verified arithmetic deductions from unverified geometric action hypotheses. We also kernel-verify the first step of an orbit-matrix argument for order 7. Assume that A satisfies the adjacency conditions and that a nontrivial permutation s with $s^7 = \mathrm{id}$ preserves A. Then the Lean proof constructs a 15×15 orbit quotient matrix in a canonical order. The proof derives the fixed vertex, the orbit structure, and the quotient equations from these hypotheses; it does not assume them. Lean does not yet prove that no such matrix exists. The Lean statement that excludes order 7 therefore still depends on `sorryAx`. For propositional logic, we implement canonical SAT encodings with Crawford symmetry-breaking predicates. A deterministic regression suite of 101 tests checks compiler components, geometric orbits, the supervisor lifecycle, positive controls, and an orbit-matrix encoder. The positive controls apply the production compilers for two actions to two known graphs with $\lambda = 1$ and $\mu = 2$: Paley(9) and the Berlekamp–van Lint–Seidel graph on 243 vertices. These checks give COMPILED status. They do not prove that a compiler is correct. CaDiCaL 1.9.5 returned `s UNSATISFIABLE` on the repaired order-7 CNF (177,097 variables, 422,508 clauses) in 58,054.87 s process time. It wrote a 26,746,488,053-byte DRAT trace. `drat-trim` reported `s VERIFIED` in 103,482.837 s (12,821,576,879 resolution steps, 378,191 core clauses, 0 RAT lemmas). That pair is PROVED. Both files are deposited in Zenodo record `10.5281/zenodo.22995101` [1]. The deposit does not decide the Thakkar–Severini CP-SAT search. The order-7 record has two certified components. They are not one certificate. Component A is in Lean. From the graph and an automorphism of order 7, the kernel derives the 12×12 exterior equations (`Z7OrbitMatrix.exterior_row_problem`). That derivation is PROVED. The kernel reports only `[propext, Quot.sound]`. It does not prove that no such matrix exists. Component B is the exterior-row CNF. It has 26,166 variables and 311,703 clauses. It is not the canonical order-7 CNF above. CaDiCaL 1.9.5 returned `s UNSATISFIABLE`, and `drat-trim` reported `s VERIFIED`. That refutation is PROVED for this CNF only. The file is not in Zenodo record `10.5281/zenodo.22995101`. The

implication from `Z7OrbitMatrix` to the exterior-row CNF is PENDING. The DRAT proof is not checked inside the Lean kernel. `z7_orbit_matrix_nonexistence` still contains `sorry`. The note also records the column-weight identity $CC^T = 10I + 2J$ for the open involution case $f = 1$, with corrected branch supports, and a 12.5-day CaDiCaL campaign that accumulated more than 2.4 billion conflicts without a satisfying assignment or a refutation. All code, Lean modules, and test suites are available at https://github.com/antoniomachuca/conway-99-graph.

Keywords Conway 99-graph · strongly regular graphs · interactive theorem proving · Lean 4 · SAT solving · DRAT certification · symmetry breaking

1 Introduction and Literature Attribution

A strongly regular graph $\mathrm{srg}(v, k, \lambda, \mu)$ is a regular graph on v vertices with degree k. Every pair of adjacent vertices has exactly λ common neighbors. Every pair of distinct non-adjacent vertices has exactly μ common neighbors. Whether a strongly regular graph with parameters $(99, 14, 1, 2)$ exists remains open [2, 3]. Conway included the question among five problems carrying a $1,000 prize. The circulated statement is the prize list *Five $1,000 Problems (Update 2017)* [4], Problem 2. Bill Cheswick added an update in 2014. Cheswick and Sloane added a postscript in 2017. That document is not a 1970 research article and it does not identify itself as conference proceedings.

Let $A \in \{0, 1\}^{99\times 99}$ denote the adjacency matrix of a Conway 99-graph. The strongly regular parameters require A to satisfy the matrix equation

$$A = A^T, \qquad \mathrm{diag}(A) = 0, \qquad A^2 + A = 12I_{99} + 2J_{99}, \tag{1.1}$$

where I_n is the $n \times n$ identity matrix and J_n is the $n \times n$ all-ones matrix. Spectral graph theory gives the eigenvalues of A: $k = 14$ (multiplicity 1), $r = 3$ (multiplicity 54), and $s = -4$ (multiplicity 44):

$$\mathrm{Spec}(A) = \{14^1, 3^{54}, (-4)^{44}\}. \tag{1.2}$$

For each vertex $x \in V(G)$, let $\Gamma(x)$ denote its open neighborhood. Let $\Gamma_2(x) = V(G) \setminus (\{x\} \cup \Gamma(x))$ denote the second subconstituent. Because $k = 14$ and $\lambda = 1$, each vertex in $\Gamma(x)$ has exactly one neighbor in $\Gamma(x)$. The induced subgraph $G[\Gamma(x)]$ is therefore a 1-regular graph of seven disjoint edges ($7K_2$). The second subconstituent $\Gamma_2(x)$ contains $99 - 1 - 14 = 84$ vertices. Because $\mu = 2$, each vertex $y \in \Gamma_2(x)$ is adjacent to exactly two vertices in $\Gamma(x)$ [2, 3].

1.1 Literature Attribution of Automorphism Group Bounds

An unconstrained combinatorial search on a 99-vertex graph involves $2^{\binom{99}{2}} = 2^{4851}$ possible edge configurations. Computational searches in this note therefore restrict attention to graphs that admit a nontrivial automorphism. Such a restriction cannot find a rigid graph. Prior literature establishes strict bounds on the automorphism group $\mathrm{Aut}(G)$:

1. **Involutions (order 2) fix exactly one vertex.** Makhnev and Minakova (2004) [5] classified prime-order automorphisms of strongly regular graphs with $\lambda = 1$ and $\mu = 2$. Makhnev's 2009 slides [6], attributing the result to Makhnev–Minakova, state that an involution of a Conway 99-graph fixes exactly one vertex.
2. **Automorphisms of order 3 are fixed-point-free; orders 6 and 9 are excluded.** Behbahani and Lam (2011) [7] proved that an automorphism of order 3 on $\mathrm{srg}(99, 14, 1, 2)$ has no fixed points. It acts as a fixed-point-free (FPF) permutation with 33 cycles of length 3. Crnković and Maksimović (2020) [8] proved that $\mathrm{Aut}(G)$ contains no subgroups of order 6 or 9. This result excludes $\mathbb{Z}_6$, S_3, $\mathbb{Z}_9$, and $\mathbb{Z}_3 \times \mathbb{Z}_3$.
3. **Prime orders $p \geq 5$ are excluded.** Behbahani and Lam (2011) [7] showed that the only possible prime divisors of $|\mathrm{Aut}(G)|$ are 2 and 3. They excluded automorphisms of order 5, 7, and higher primes through computer-assisted orbit matrix enumeration. Cesarz and Woldar (2025) [3] gave computer-free proofs that:
$$2 \mid |\mathrm{Aut}(G)| \implies |\mathrm{Aut}(G)| \mid 6, \qquad 7 \mid |\mathrm{Aut}(G)| \implies \mathrm{Aut}(G) \cong \mathbb{Z}_7. \tag{1.3}$$
Combining this even-order implication with the exclusion of order 6 gives: if $2 \mid |\mathrm{Aut}(G)|$, then $|\mathrm{Aut}(G)| = 2$. Therefore, $\mathrm{Aut}(G)$ contains no elements of order 4 or 6. This is a consequence of the cited papers, not a new group-order theorem.
4. **Computational benchmark context.** Thakkar and Severini (2026) [9] encode forced-structure reductions and prescribed automorphisms in Google OR-Tools CP-SAT. They report that the single-fixed-point order-7 instance returned `UNKNOWN` after 48 hours on 14 cores. Their paper still treats a $\mathbb{Z}_7$-symmetric graph as open. A CNF refutation is a certificate for that instance only when the two formulations are shown to define the same set of graphs. This note does not give that correspondence.

1.2 Contributions and Scope of this Benchmark Report

The note keeps the following items separate. The primary contents are:

- We introduce a four-state taxonomy (PROVED, COMPILED, EXPLORED, PENDING) to clarify proof verification and modeling limits.
- In Lean 4, we verify structural invariants (K_4-freeness, 1-factor neighborhood decomposition, diameter at most 2) and the decidability of the adjacency conditions. The Lean kernel checks these proofs using only `[propext, Quot.sound]`. We also formalize the arithmetic contradictions that arise in character-theoretic analyses.
- In Lean 4, we prove that a Conway 99-graph with a nontrivial automorphism s such that $s^7 = \mathrm{id}$ gives a canonical 15×15 orbit quotient matrix (`conway_z7_induces_orbit_matrix`). The kernel reports only `[propext, Quot.sound]`. The orbit structure is known from the literature [7, 3]. The contribution is a mechanized derivation. The nonexistence of such a matrix remains PENDING in Lean. `Z7OrbitMatrix.exterior_row_problem` derives the 12×12 exterior equations from `Z7OrbitMatrix`. That step is PROVED and does not prove that no matrix exists.
- We implement canonical SAT compilers with Crawford symmetry-breaking predicates [10]. A 101-test regression suite checks helper truth tables, the $f = 1$ coordinate model, the bounded-run supervisor, positive controls, and an orbit-matrix encoder. The positive controls apply the production compilers for the involution with one fixed vertex and for the fixed-point-free action of order 3 to Paley(9) and to the Berlekamp–van Lint–Seidel graph. These tests do not certify that a historical CNF matches the graph. They do not cover the order-7 compiler. A separate file has four COMPILED tests of the fixed-3 encoding. The 101-test count does not include those four tests. Those tests are not the fixed-point exclusion of Crnković and Maksimović.
- The repaired compiler's order-7 CNF has a CaDiCaL 1.9.5 log ending `s UNSATISFIABLE` and a `drat-trim` log ending `s VERIFIED` on that file, so the formula refutation is PROVED. The CP-SAT model of Thakkar and Severini is a separate encoding.
- The exterior-row CNF is a second formula, with 26,166 variables and 311,703 clauses. CaDiCaL 1.9.5 returned `s UNSATISFIABLE`, and `drat-trim` reported `s VERIFIED`. The refutation is PROVED for that CNF only. The implication from `Z7OrbitMatrix` to the CNF is PENDING, and `z7_orbit_matrix_nonexistence` still contains `sorry`.
- We prove that every column of a binary 7×42 matrix satisfying $CC^T = 10I_7 + 2J_7$ has weight 2, and we correct the support labels of the three $f = 1$ branch representatives. A 12.5-day campaign on the $f = 1$ and fixed-point-free order-3 encodings produced no satisfying assignment and no refutation.

2 Lean 4 Formalization and Operational Taxonomy

To prevent ambiguity between mechanized proofs, empirical solver outputs, and open conjectures, we classify every result and artifact into one of four operational states:

PROVED. A theorem verified by the Lean 4 kernel using only the foundational axioms `[propext, Quot.sound]`; or a SAT claim for which the cited CNF is the file that was checked, its solver log ends in `s UNSATISFIABLE`, and `drat-trim` reports `s VERIFIED` on that pair. A checker log for a different formula does not meet this state.

COMPILED. Formally typed Lean 4 code compiling with `sorryAx` stubs for open subgoals; or Python SAT compilers whose clausal outputs pass deterministic regression tests.

EXPLORED. Combinatorial searches, matrix calculations, and multi-threaded solver executions that finished without finding a satisfying assignment or producing a proof certificate.

PENDING. Open mathematical questions (such as graph existence or full rigidity $|\mathrm{Aut}(G)| = 1$) and unfinished formalizations connecting group actions to arithmetic hypotheses.

2.1 Kernel-Verified Structural Properties in Lean 4

We formalize the adjacency definition of a Conway 99-graph in Lean 4 [11] with the predicate `ConwayAdj`. This predicate enforces symmetry, zero diagonal, binary entries, and the matrix equation (1.1). To support verified computation,

we define a computable Boolean function `checkConway` and prove its equivalence to `ConwayAdj`:

$$\texttt{conway_soundness}, \texttt{conway_complete}: \quad \forall A \in \mathbb{N}^{99\times 99}, \quad \texttt{checkConway}(A) = \texttt{true} \iff \texttt{ConwayAdj}(A). \tag{2.1}$$

In the modules `Structural.lean` and `Z2Classification.lean`, we prove structural invariants directly from `ConwayAdj`:

- `conway_k4_free` (`Structural.lean`): Every graph satisfying the matrix equation contains no complete subgraph K_4, because $\lambda = 1$.
- `conway_neighborhood_one_factor` (`Structural.lean`): For each vertex x, the open neighborhood $G[\Gamma(x)]$ is a 1-regular graph of seven disjoint edges ($7K_2$).
- `conway_diameter_at_most_two` (`Structural.lean`): Any two non-adjacent vertices have common neighbors because $\mu = 2 > 0$. Therefore, $\mathrm{diam}(G) \leq 2$.
- `conway_z2_f3_fixed_points_dichotomy` (`Z2Classification.lean`): If an involution fixes three vertices, their induced subgraph is either a triangle K_3 or an independent set $3K_1$.

The Lean kernel checked all these declarations with `#print axioms`. They depend solely on `[propext, Quot.sound]`.

2.2 The Boundary Between Arithmetic Refutations and Graph Hypotheses

Character-theoretic analyses of automorphisms impose numerical constraints on fixed edge counts and eigenspace multiplicities. In the Lean library, we kernel-verify the arithmetic refutations of these systems:

- For an involution with $f = 5$ fixed vertices, the declaration `conway_z2_f5_spectral_formula_contradiction` refutes the existence of an integer ε_1 satisfying:
$$(\varepsilon_1 = 18 \vee \varepsilon_1 = 15) \wedge (\varepsilon_1 \bmod 7 = 6). \tag{2.2}$$
- For $f = 7$, `conway_no_z2_f7_formula_automorphism` refutes:
$$\varepsilon_1 \in \{7, 10, 13\} \wedge (\varepsilon_1 \bmod 7 = 2). \tag{2.3}$$
- In `CesarzWoldarTheorems.lean`, terminal arithmetic contradictions such as $7a = 62$ or sums of even integers equaling 5 are verified using only `[propext, Quot.sound]`.

These arithmetic refutations are PROVED. However, their wrappers (such as `conway_no_z2_f5_automorphism`) take the counting identities and spectral congruences as hypotheses. Deducing these arithmetic hypotheses directly from `ConwayAdj(A)` and an involution t in Lean remains PENDING. Likewise, the classification theorem `conway_automorphism_group_restricted` uses `sorryAx` for open subcases, placing its status at COMPILED.

2.3 Kernel-Checked Reduction from an Order-7 Action to the Orbit Quotient

Behbahani and Lam [7] excluded order 7 with orbit matrices. The Lean library now contains a kernel-checked form of the first step of such an argument: the passage from a graph and an automorphism to a quotient matrix.

Let s act on $V(G)$ with orbits $O_0, \ldots, O_{m-1}$ of sizes $n_i = |O_i|$. For $u \in O_i$, let B_{ij} be the number of neighbors of u in O_j. Because s preserves adjacency, B_{ij} does not depend on the choice of u. The Lean structure `Z7OrbitMatrix` requires $m = 15$, $n_0 = 1$, and $n_i = 7$ for $i \geq 1$. It also requires these conditions for all indices i and j:

1. $\sum_j B_{ij} = 14$, $n_i B_{ij} = n_j B_{ji}$, and $(B^2)_{ij} + B_{ij} = 12\,\delta_{ij} + 2\,n_j$.
2. $B_{ij} \leq n_j$ and $B_{ii} \in \{0, 2\}$. If $i, j \geq 3$, then $B_{ij} \leq 4$.
3. $B_{0j} = 7$ if $j \in \{1, 2\}$, and $B_{0j} = 0$ otherwise. $B_{i0} = 1$ if $i \in \{1, 2\}$, and $B_{i0} = 0$ otherwise.
4. $B_{11} = B_{22} = 0$ and $B_{12} = B_{21} = 1$.
5. The pair (B_{i1}, B_{i2}) is $(2, 0)$ for $3 \leq i \leq 5$, $(0, 2)$ for $6 \leq i \leq 8$, and $(1, 1)$ for $9 \leq i \leq 14$.

Theorem 2.1 (`conway_z7_induces_orbit_matrix`; PROVED)**.** *Let A satisfy `ConwayAdj`. Let s be a bijection of the vertex set $\{0, \ldots, 98\}$ such that $s^7 = \mathrm{id}$, $s \neq \mathrm{id}$, and $A_{s(u)\,s(v)} = A_{uv}$ for all vertices u and v. Then a matrix that satisfies `Z7OrbitMatrix` exists.*

The Lean statement has no hypotheses other than those in Theorem 2.1. It does not take an orbit partition, an orbit count, or a quotient matrix as input. The proof has these steps:

1. `Z7Cycles.lean`: each orbit of s has size 1 or 7. The number of fixed vertices is congruent to 99 modulo 7. Thus at least one fixed vertex x_0 exists.
2. `GraphLocalCounts.lean`: `ConwayAdj` gives degree 14, one common neighbor for adjacent vertices, and two common neighbors for distinct nonadjacent vertices. The unique common neighbor defines a perfect matching on each neighborhood.
3. `Z7FixedGeometry.lean` and `Z7GraphFrame.lean`: s moves every neighbor of x_0. The set $\Gamma(x_0)$ is the union of two orbits of size 7, and no edge joins two vertices of the same such orbit. The vertex x_0 is the only fixed vertex.
4. `Z7OrbitEnumeration.lean`: a computable list contains one representative of each orbit. The list has 15 entries, because one orbit has size 1 and $(99-1)/7 = 14$ orbits have size 7. Indices 0, 1, and 2 hold x_0 and the two orbits in $\Gamma(x_0)$.
5. `OrbitQuotient.lean` and `OrbitReindex.lean`: the matrix B is well defined. It satisfies the three equations of condition 1 and the bound $B_{ij} \le n_j$. If n_i is odd, then B_{ii} is even.
6. `Z7QuotientBoundary.lean` and `Z7RootedConstruction.lean`: conditions 3 and 4 follow from the structure of $\Gamma(x_0)$.
7. `Z7CanonicalArithmetic.lean`: for $i \ge 3$, the quotient equation gives $B_{i1} + B_{i2} = 2$. The equations then force exactly three outer orbits of type $(2,0)$, three of type $(0,2)$, and six of type $(1,1)$. They also give $B_{ii} \in \{0,2\}$, and $B_{ij} \le 4$ for $i,j \ge 3$. An explicit permutation puts the outer orbits in the order of condition 5.

For Theorem 2.1, `#print axioms` reports `[propext, Quot.sound]`. The new modules contain no `sorry`, no `native_decide`, and no new axioms. The file `TestZ7Canonical.lean` computes orbit representatives for two finite permutations: a permutation of 99 points with one fixed point and fourteen 7-cycles, which gives 15 representatives, and the identity map, which gives 99. These checks test the definitions. They are not Conway graphs.

The theorem gives necessary conditions only. It does not state that a matrix with these properties comes from a graph. `Z7OrbitMatrix.exterior_row_problem`, in `Conway/Z7ExteriorBlock.lean`, derives the 12×12 exterior equations from `Z7OrbitMatrix`. For that statement, `#print axioms` reports `[propext, Quot.sound]`. The derivation does not use the DRAT proof, and it does not prove that no matrix exists. The statement `z7_orbit_matrix_nonexistence` is still a `sorry`. `conway_no_z7_automorphism` uses it and remains COMPILED. The DRAT certificate of the exterior-row CNF is not checked inside the kernel.

3 Canonical SAT Encodings and Symmetry Breaking

To explore automorphism classes computationally, we map the graph constraints under a cyclic automorphism group $\langle g \rangle \cong \mathbb{Z}_p$ into Conjunctive Normal Form (CNF).

3.1 Orbit Parameterization and Crawford Symmetry Cuts

Under the action of an automorphism g of prime order p, the 99 vertices partition into m orbits $\mathcal{O}_1, \dots, \mathcal{O}_m$, where $|\mathcal{O}_i| \in \{1, p\}$. The adjacency between two orbits of size p is invariant under cyclic shifts. A circulant Boolean vector $c_{i,j} \in \{0,1\}^p$ parameterizes this adjacency, where $c_{i,j,k} = 1$ if vertex $0 \in \mathcal{O}_i$ connects to vertex $k \in \mathcal{O}_j$.

The strongly regular graph conditions translate into exact cardinality constraints:

$$\sum_{u \in \Gamma(x)} A_{u,v} = \begin{cases} 14 & \text{if } x = v, \\ 1 & \text{if } x \sim v, \\ 2 & \text{if } x \not\sim v. \end{cases} \tag{3.1}$$

We convert these pseudo-Boolean constraints to CNF with Tseitin variables. Exact cardinality constraints use PySAT sequential counters (`EncType.seqcounter`). The compilers used for this note do not emit sorting networks.

To prune isomorphic search spaces, we enforce symmetry-breaking predicates following Crawford et al. [10]. Let $\mathcal{G} \le \mathrm{Sym}(m)$ be the automorphism group preserving the orbit partition. For each non-identity generator $\pi \in \mathcal{G}$, we enforce a lexicographical leader constraint: the assignment vector X must precede its permuted image $\pi(X)$:

$$X \le_{\text{lex}} \pi(X). \tag{3.2}$$

For variables $(x_1, \ldots, x_n)$ and their images $(y_1, \ldots, y_n) = (\pi(x_1), \ldots, \pi(x_n))$, we introduce auxiliary variables $e_i \leftrightarrow (x_i = y_i)$ and enforce:

$$(\neg x_i \wedge y_i) \vee (e_i \wedge (X_{i+1..n} \leq_{\text{lex}} Y_{i+1..n})). \tag{3.3}$$

3.2 Compiler Verification and Defect Repair

Translating graph constraints into propositional clauses requires careful validation. In prototype implementations, primary Boolean variables were 1-indexed (with ID 1 for x_1), while helper functions such as `get_and_lit` and `add_card_equals` treated integer `1` as the Boolean constant `True`. When encoding $x_1 + x_2 = 1$ via `add_card_equals([1, 2], 1)`, the compiler evaluated literal 1 as constant `True`. This produced the degenerate unit clause `[-2]`, which enforced $x_2 = 0$ while omitting x_1. The formula then admitted $(x_1, x_2) = (0, 0)$ and rejected $(0, 1)$.

Example 3.1 (The Literal-1 Collision Defect)**.** The collision of variable ID 1 with the Boolean constant `True` changes an exact-1 constraint on $\{x_1, x_2\}$ into the unit clause $\neg x_2$. This defect shows why compiler primitives require end-to-end truth table tests.

We refactored the compiler architecture to separate variable indices from Boolean constants. Variables are now allocated through explicit Tseitin gate managers, reserving index 0 as an internal sentinel.

To test the compilers, we keep a deterministic regression suite of 101 tests in `tests/`:

1. **Compiler-Helper Unit Tests (20 tests):** Verifies truth tables for conjunctions, cardinality constraints, circulant orbit variables, and Crawford lex-leader cuts for Z_7, Z_3 fixed-3, and Z_3 FPF compilers.
2. **Geometric and Input-Audit Tests (11 tests):** Evaluates 2,394 cardinality constraints symbolically against the coordinate model, checking branch cut consistency and index mappings.
3. **Supervisor Process Tests (8 tests):** Validates bounded execution, process lifecycle management, wall-time enforcement, and proof byte limits.
4. **Orbit-Pair Tests (5 tests):** Check the orbit-pair enumeration for the case $f = 1$.
5. **Positive-Control Tests (32 tests):** Run the production positive and negative controls of Section 3.3 on Paley(9), check the Berlekamp–van Lint–Seidel construction, and test the clause-check harness. This group also keeps older tests of standalone toy encoders, coordinate geometry, and relaxed subsystems. Those older tests do not show that a complete production formula admits a known graph.
6. **Orbit-Matrix Tests (25 tests):** Check the quotient-level order-7 encoder against small exhaustive cases and known quotients (10 tests), and check the run supervisor for that encoder (15 tests). The supervisor tests check model validation, checker integrity, and the treatment of `UNKNOWN` results.

On 3 October 2026 all 101 tests passed on the integrated revision. Passing tests support COMPILED status for the current compiler sources. They do not rewrite CNF files generated earlier, and they do not prove that those files express the intended graph constraints.

3.3 Positive Controls on the Production Compilers

An `s UNSATISFIABLE` result cannot show that an encoding admits all graphs that it must admit. The literal-1 defect of Example 3.1 made a formula too strong. A positive control applies the same compiler to a known graph with the same λ and μ. If the compiler removes that graph, the control fails.

Two production compilers now accept the vertex degree as an argument: the compiler for an involution with one fixed vertex (`build_z2_f1_cnf.py`) and the compiler for a fixed-point-free automorphism of order 3 (`scripts/build_z3_canonical_cnf.py`). The order-3 compiler also accepts the number of orbits. The values $\lambda = 1$ and $\mu = 2$ stay fixed. With the default arguments, the outputs of both compilers did not change: the SHA-256 digests agree with the outputs of the unmodified compiler sources. The default formulas have $666{,}309$ variables and $1{,}508{,}157$ clauses for the involution, and $852{,}324$ variables and $1{,}860{,}024$ clauses for order 3. This comparison does not use the CNF files of earlier runs.

Two known graphs with $\lambda = 1$ and $\mu = 2$ are the controls:

- Paley(9), which is $\mathrm{srg}(9, 4, 1, 2)$. For each of the two actions, the solver finds a model of the complete compiled formula. The decoded graph satisfies the strongly regular conditions, and the model satisfies all clauses.

- The Berlekamp–van Lint–Seidel graph, which is $\mathrm{srg}(243, 22, 1, 2)$ [12]. The script builds it as a Cayley graph on $\mathbb{F}_3^5$ from the description in [13], and it checks $A^2 + A = 20I + 2J$ before use. The map $x \mapsto -x$ is an involution with one fixed vertex. A translation is a fixed-point-free automorphism of order 3. The known graph sets the primary variables. The check then finds values for the auxiliary variables such that all emitted clauses are true: 28,257,515 clauses for the involution and 27,424,360 clauses for order 3. The auxiliary variables are checked in disjoint batches, and the check rejects the reuse of an auxiliary variable from an earlier batch.

Negative controls also pass. If one primary value of the Paley(9) assignment is changed, the compiled formula becomes unsatisfiable. A truth-table test checks the literal-1 case of Example 3.1 in both production compilers.

These controls are COMPILED. They do not prove that a compiler is correct for all inputs, and each Berlekamp–van Lint–Seidel check uses one fixed graph. They do not cover the order-7 compiler of Section 4 or the order-3 compiler with three fixed vertices. No DRAT proof is involved. The two Berlekamp–van Lint–Seidel checks are not part of the default test run. The repository records their output in `tests/production_positive_control_results.json`.

A separate file has four tests of the fixed-3 encoding. The file is `tests/test_z3_fixed3_encoding_coverage.py`. The four tests passed. The four tests are COMPILED. The tests check 54 fixed one-factor entries. The tests check the circulant adjacency literal. The tests check 148 degree cardinalities. The tests check 1520 common-neighbor pair orbits. Those tests call no solver on the Conway fixed-3 formula. `conway_no_z3_fixed3_automorphism` still contains `sorry`. These tests do not cover the order-7 compiler. These tests do not cover the fixed-point-free order-3 encoding. Crnković and Maksimović (2020) already exclude this fixed-point case. These tests are not that exclusion. The 101-test count does not include these four tests.

4 Certified Order-7 CNF Refutation

Behbahani and Lam (2011) [7] excluded automorphisms of order 7 by a computer-assisted orbit-matrix argument. Cesarz and Woldar (2025) [3] proved, without a computer, that $7 \mid |\mathrm{Aut}(G)|$ implies $\mathrm{Aut}(G) \cong \mathbb{Z}_7$. Thakkar and Severini (2026) [9] nevertheless report a single-fixed-point $\mathbb{Z}_7$ CP-SAT run that remained `UNKNOWN` after 48 hours on 14 cores, and they describe that symmetric case as still open. Here we report a verified SAT refutation for the canonical order-7 encoding.

4.1 Encoding Parameters and Formula Statistics

Under a cyclic automorphism g of order 7, the 99 vertices partition into:

- Exactly 1 fixed vertex x_0;
- 2 orbits of size 7 in the neighborhood $\Gamma(x_0)$ (because $k = 14$);
- 12 orbits of size 7 in the second subconstituent $\Gamma_2(x_0)$ (84 vertices).

This decomposition matches the orbit structure of Cesarz and Woldar [3]. The canonical CNF encoding uses 498 primary variables: 36 internal orbit valences and 462 cross-orbit circulant variables. We encode degree, $\lambda = 1$, and $\mu = 2$ constraints into CNF and add Crawford lex-leader cuts for the 11 non-identity elements of the stabilizer $G_{\mathbb{Z}_7}$. The resulting formula emitted by the repaired compiler has:

$$\text{Variables: } 177{,}097, \qquad \text{Clauses: } 422{,}508. \tag{4.1}$$

4.2 Solver Execution and Independent DRAT Verification

The formula was solved with local CaDiCaL 1.9.5 [14]. That log records header `p cnf 177097 422508`, verdict `s UNSATISFIABLE`, exit 20, 58,054.87 s process time, 58,595.63 s real time, and 119,353,667 conflicts, and it closed a 26,746,488,053-byte proof at `/Volumes/Untitled/conway_repro_z7/proof_z7_canonical.drat`. The binary version string 3.0.1 and commit `c60730422e758ef1cebe7aeddf2dda31c996bf04` belong to an earlier run on the formula `p cnf 176613 421562`. The following lines are a non-contiguous extract from the 1.9.5 log:

```
c DRAT proof file '/Volumes/Untitled/conway_repro_z7/proof_z7_canonical.drat'
    closed
c DRAT 127774798 added clauses 50.10%
c DRAT 127286510 deleted clauses 49.90%
c DRAT 26746488053 bytes (25507.44 MB)
s UNSATISFIABLE
c total process time since initialization:     58054.87    seconds
```

```
c total real time since initialization:        58595.63    seconds
c exit 20
```

drat-trim [15] logged backward checking of this CNF against that proof and s VERIFIED after 103,482.837 s verification time. The command-line timeout is not in the log:

```
c parsing input formula with 177097 variables and 422508 clauses
c finished parsing, read 26746488053 bytes from proof file
c detected empty clause; start verification via backward checking
c 378191 of 422508 clauses in core
c 66748552 of 127774799 lemmas in core using 12821576879 resolution steps
c 0 RAT lemmas in core; 62829297 redundant literals in core lemmas
s VERIFIED
c verification time: 103482.837 seconds
```

The checker extracted 378,191 core clauses (89.51%) and 66,748,552 core lemmas (52.24%) across 12,821,576,879 resolution steps, requiring zero RAT lemmas (pure RUP resolution).

Table 1: Two order-7 encodings. No equivalence between the CP-SAT model and this CNF is proved.

Source	Object	Tool	Runtime	Outcome
Thakkar–Severini [9]	CP model	CP-SAT	$>$ 48 h (14 cores)	UNKNOWN
This work (solve)	p cnf 177097 422508	CaDiCaL 1.9.5	58,054.87 s	s UNSATISFIABLE
This work (verify)	24.91 GiB DRAT	drat-trim	103,482.84 s	s VERIFIED

4.3 Cryptographic Integrity and Operational Status

Both artifacts are preserved with deterministic SHA-256 cryptographic checksums:

- conway_z7_canonical.cnf:
  fc67a85f300151b628ffc5d808b5cb384e69efac2a87c9891aab2c8115e0af47
- proof_z7_canonical.drat:
  f312d2a83d85bd11f1c9c2428cb67566ca12b11c70527cfebf0f72572da14a39

Those two digests match the CNF and the 26,746,488,053-byte proof in the Zenodo record 10.5281/zenodo.22995101 [1]. The earlier record 10.5281/zenodo.22983776 stores this CNF beside a 192,218,491-byte proof. Under our taxonomy:

1. The propositional refutation of conway_z7_canonical.cnf is PROVED: the solver log ends in s UNSATISFIABLE and drat-trim reports s VERIFIED on that pair.
2. The graph-to-CNF step is COMPILED. The regression tests pass and do not kernel-certify the encoding. A Lean equivalence remains PENDING.
3. This is a checked refutation of this CNF. It does not re-prove the Behbahani–Lam exclusion [7], and it does not decide the Thakkar–Severini CP-SAT instance [9].

4.4 Exterior-Row CNF

This subsection is component B. It is not component A, and it is not the canonical order-7 CNF. The exterior-row CNF is not the canonical order-7 CNF. The file name is z7_row_problem.cnf. The directory is /Volumes/Untitled/conway_z7_row_direct_long_20261003_01. The SHA-256 digest of z7_row_problem.cnf is b746471acd527e52da4706a354fc6fd0d225bb638c121f8b7ec6f965ec5a4ee4. The exterior-row CNF has 26,166 variables. The exterior-row CNF has 311,703 clauses. scripts/z7_row_problem_cnf.py emits the exterior-row CNF. The script uses direct encoding. The exterior-row CNF states the 12 by 12 exterior row problem. The exterior-row CNF adds no Frob(21) block cuts. The exterior-row CNF adds no orbit-relabeling cuts. The exterior-row CNF identifies C_{ij} with C_{ji}. The predicate requires this symmetry. CaDiCaL 1.9.5 ran on the exterior-row CNF. solver.log contains s UNSATISFIABLE. solver.log records exit 20. The solver used 9,051.98 seconds of real time. The solver reported 32,331,971 conflicts. The checker read 4,750,239,705 bytes from the proof file. checker.log contains s VERIFIED. The checker finished in 8,879.590 seconds. The core

contains 201,494 of 311,703 clauses. The core contains 12,551,572 of 56,577,564 lemmas. The checker counted 1,425,060,916 resolution steps. The core contains 0 RAT lemmas. The checker printed duplicate-literal warnings. The verdict line is `s VERIFIED`. This refutation is PROVED for the exterior-row CNF only. The exterior-row CNF is not in Zenodo record `10.5281/zenodo.22995101`. The exterior-row proof is not in that Zenodo record. The file `tests/test_z7_row_problem_cnf.py` contains five tests. The five tests run the encoder on small exhaustive instances. That test file is COMPILED evidence. The tests do not prove that every integer solution of the Lean predicate is a model of the exterior-row CNF.

4.5 Relation to the Lean Reduction and a Quotient-Level Formula

Theorem 2.1 ends at the quotient matrix. The canonical order-7 CNF uses graph-level circulant variables. No Lean statement connects the quotient matrix to the canonical order-7 CNF. No Lean statement connects the graph to the canonical order-7 CNF. The graph-to-CNF step for the canonical order-7 CNF remains COMPILED. The positive controls of Section 3.3 do not cover the canonical order-7 compiler.

The repository also contains a CNF encoding of 15-orbit quotient conditions (`scripts/orbit_matrix_encoding.py`, model `z7-general`). This formula has 397,296 variables and 806,262 clauses. A bounded CaDiCaL 1.9.5 run with a 120 s limit stopped with `c UNKNOWN` after 120.22 s real time and 81.18 s process time. The log has no terminal `s` line, so we did not run a checker. The partial proof file is not a certificate. This run is EXPLORED. A proof that every `Z7OrbitMatrix` gives a model of this formula is PENDING. That formula is not the exterior-row CNF. The two certified components stay separate. Component A, the derivation of the exterior equations, is PROVED in Lean. Component B, the exterior-row CNF refutation, is PROVED for that CNF only. The implication from `Z7OrbitMatrix` to the exterior-row CNF is PENDING. The kernel does not check the DRAT proof. `z7_orbit_matrix_nonexistence` still contains `sorry`, so `conway_no_z7_automorphism` remains COMPILED. The 397,296-variable run remains EXPLORED.

The repository also keeps an older quotient formula with extra block restrictions. These restrictions come from the Frobenius group $\mathbb{Z}_7 \rtimes \mathbb{Z}_3$ of order 21, and they do not follow from an automorphism of order 7. `drat-trim` verified a refutation of that exact formula. We make no claim about graphs from it.

5 Incidence Geometry and Search Limits for Open Branches

The literature cited above leaves two prime-order actions unresolved by this note:

1. An involution ($p = 2$) with exactly $f = 1$ fixed point (Makhnev–Minakova);
2. A fixed-point-free automorphism of order 3. Behbahani's fixed-point restriction does not remove it, and Crnković–Maksimović do not exclude it when they exclude orders 6 and 9.

5.1 Incidence Geometry of the One-Fixed-Vertex Involution ($f = 1$)

Let $t \in \mathrm{Aut}(G)$ be an involution with $\mathrm{Fix}(t) = \{x_0\}$. The rooted neighborhood $\Gamma(x_0)$ consists of 7 matched pairs under t. The second subconstituent $\Gamma_2(x_0)$ contains 84 vertices partitioned into 42 transposed pairs (orbits $O_0, \ldots, O_{41}$).

Each orbit $O_j = \{u, t(u)\}$ has a support in $\Gamma(x_0)$. This support is the set of matched pairs in $\Gamma(x_0)$ containing neighbors of u. We define the binary support-incidence matrix $C \in \{0,1\}^{7\times 42}$, where $C_{i,j} = 1$ if orbit O_j connects to the i-th matched pair of $\Gamma(x_0)$. By construction, $C = [C_1 \mid C_1]$, where $C_1 \in \{0,1\}^{7\times 21}$ is the vertex-edge incidence matrix of K_7. Computing the product gives the matrix equation:

$$CC^T = 10I_7 + 2J_7. \tag{5.1}$$

Proposition 5.1 (Exact Column Weight Distribution)**.** *For every binary matrix $C \in \{0,1\}^{7\times 42}$ satisfying $CC^T = 10I_7 + 2J_7$, each column has weight exactly 2.*

Proof. Let $k_j = \sum_{i=1}^{7} C_{i,j}$ be the sum of column j. The total number of ones in C is $\sum_{j=1}^{42} k_j = \mathrm{Tr}(CC^T) = \mathrm{Tr}(10I_7 + 2J_7) = 70 + 14 = 84$. The sum of squares of column sums equals the quadratic form:

$$\sum_{j=1}^{42} k_j^2 = \mathbf{1}_7^T(CC^T)\mathbf{1}_7 = \mathbf{1}_7^T(10I_7 + 2J_7)\mathbf{1}_7 = 10(7) + 2(7^2) = 70 + 98 = 168. \tag{5.2}$$

The variance of k_j around 2 is:

$$\sum_{j=1}^{42}(k_j - 2)^2 = \sum_{j=1}^{42} k_j^2 - 4\sum_{j=1}^{42} k_j + 4(42) = 168 - 4(84) + 168 = 168 - 336 + 168 = 0. \tag{5.3}$$

Because $(k_j - 2)^2 \geq 0$ for all j, we have $k_j = 2$ for all $j \in \{1, \ldots, 42\}$. □

Note that $\mathrm{Tr}(C^T C) = \mathrm{Tr}(CC^T) = 84$, while the quadratic form $\mathbf{1}_7^T (CC^T)\mathbf{1}_7 = 168$ gives the sum of squared column weights.

5.2 Stabilizer Decomposition and Branching Classes

The binary matrix C allows independent swaps of its 21 duplicate-column pairs. This generates a relabeling group $(S_2)^{21} \rtimes S_7$ of order $10,569,646,080$. However, in the signed rooted coordinate system of the graph, the valid coordinate symmetry group is:

$$W = (\mathbb{Z}_2)^7 \rtimes S_7, \qquad |W| = 2^7 \cdot 7! = 645,120. \tag{5.4}$$

Fixing a distinguished orbit O_0 with support $\{0, 1\}$, the stabilizer $\mathrm{Stab}_W(O_0)$ has order 15,360 inside W. The global sign flip fixes every orbit label, so it lies in the kernel of the action on the 42 labels. The induced permutation group therefore has order 7,680, and the faithful stabilizer of O_0 has order 7,680. These orders were obtained by enumerating the stabilizer in the signed coordinate model. Under $\mathrm{Stab}_W(O_0)$, the remaining 41 orbits fall into three classes:

- **Class A (opposite phase, identical support):** Size 1, representative O_{21}, support $\{0, 1\}$, phase $-$. The abstract residual order is 15,360 and the faithful residual order is 7,680.
- **Class B (intersecting support):** Size 20, representative O_1, support $\{0, 2\}$, phase $+$. Then $|\{0, 1\} \cap \{0, 2\}| = 1$. The abstract residual order is 768 and the faithful residual order is 384.
- **Class C (disjoint support):** Size 20, representative O_{11}, support $\{2, 3\}$, phase $+$. Then $\{0, 1\} \cap \{2, 3\} = \emptyset$. The residual orders are again 768 and 384.

An earlier draft assigned support $\{1, 6\}$ to O_1. That label is the support of O_{10}. Since O_0 has support $\{0, 1\}$, the index O_{10} is another intersecting representative, in the same class as O_1, and it is not a disjoint representative. The representatives used here are $A = 21$, $B = 1$, and $C = 11$. The input-audit tests check these indices against the compiler's edge list of K_7.

5.3 Limits of Combinatorial Search: The Distributed Solver Campaign

To test whether CDCL solvers can resolve the open branches without further theoretical reductions, we ran a multi-threaded search campaign using CaDiCaL on Google Cloud Platform (16 cores, dedicated instance). The search targeted the Z_2 ($f = 1$) branches and the Z_3 FPF encoding.

The campaign ran for 12.5 days and ended on 23 September 2026. The conflict total below is the aggregate recorded for that run. The solver logs are not part of this source, and the machines have been shut down, so a reader cannot recompute the total from this file:

- The solvers accumulated over **2.4 billion conflicts** ($> 2.4 \times 10^9$) across multiple random seeds and variable selection heuristics.
- The solvers found neither a satisfying assignment (graph witness) nor an unsatisfiable termination (refutation).

Two limits apply to the formulas of this campaign. First, the campaign started before the compiler repair of 21 September 2026, and we did not audit which compiler version wrote each formula. Some formulas can therefore contain the literal-1 defect of Example 3.1. Second, project records name O_{10} as the representative of the disjoint class for at least part of the campaign. As shown above, O_{10} is an intersecting representative. The campaign therefore may not cover the disjoint class.

Under our taxonomy, this campaign is classified as EXPLORED. Conflict counts, clause counts, and proof sizes do not measure the proportion of the mathematical search space eliminated. The 12.5-day run ended without a satisfying assignment or a refutation.

6 Conclusion

This report records the following boundary. It does not decide existence.

1. The Lean library contains kernel-checked structural invariants and arithmetic refutations. Counting hypotheses that are merely assumed remain PENDING. All Lean modules are available in the public repository.

2. Component A is PROVED. The Lean library derives the canonical 15-orbit quotient matrix from a Conway 99-graph with an automorphism of order 7 (Theorem 2.1), and `Z7OrbitMatrix.exterior_row_problem` derives the 12×12 exterior equations. Neither statement proves that no matrix exists. `z7_orbit_matrix_nonexistence` still contains `sorry`, so the Lean exclusion of order 7 remains COMPILED.
3. The current SAT compilers use Crawford symmetry breaking and sequential counters. A literal-index defect was repaired on 21 September 2026. The 101-test regression suite checks the current compiler sources. It does not repair CNF files written before that date. Positive controls on Paley(9) and on the Berlekamp–van Lint–Seidel graph exercise the production compilers for the involution with one fixed vertex and for the fixed-point-free action of order 3. They are COMPILED and do not cover the order-7 compiler. A separate file has four COMPILED tests of the fixed-3 encoding. Those tests call no solver on the Conway fixed-3 formula. Those tests are not the fixed-point exclusion of Crnković and Maksimović. The 101-test count does not include those four tests.
4. The canonical order-7 CNF from the repaired compiler is PROVED as a formula. CaDiCaL 1.9.5 returned `s UNSATISFIABLE` in 58,054.87 s process time, and `drat-trim` reported `s VERIFIED` on the 26,746,488,053-byte proof in 103,482.837 s. The SHA-256 digests of the canonical order-7 CNF and its DRAT proof match Zenodo record 10.5281/zenodo.22995101. The earlier record 10.5281/zenodo.22983776 stores a 192,218,491-byte proof from an earlier run. Component B is the exterior-row CNF, a different formula. CaDiCaL 1.9.5 returned `s UNSATISFIABLE`, and `drat-trim` reported `s VERIFIED`. That refutation is PROVED for the exterior-row CNF only. The file is not in Zenodo record 10.5281/zenodo.22995101. The implication from `Z7OrbitMatrix` to this CNF is PENDING. The kernel does not check the DRAT proof. A 120-second run on the 397,296-variable CNF ended with `c UNKNOWN`. That run is EXPLORED. The 397,296-variable CNF is not the exterior-row CNF.
5. Every binary 7×42 solution of $CC^T = 10I_7 + 2J_7$ has column weight 2. In the signed model the three $f = 1$ representatives are O_{21}, O_1 with support $\{0, 2\}$, and O_{11} with support $\{2, 3\}$. The multi-threaded search remained EXPLORED.

The Conway 99-Graph and the Rigidity Conjecture. A central question is whether any Conway 99-graph must be rigid:

Conjecture 6.1 (Rigidity Conjecture)**.** *If a strongly regular graph with parameters* $(99, 14, 1, 2)$ *exists, then* $\mathrm{Aut}(G) = \{1\}$.

A search that assumes a nontrivial automorphism (Z_2, Z_3, or Z_7) excludes every rigid graph from its search space. Refuting the open $f = 1$ and fixed-point-free order-3 branches would prove the rigidity conjecture and would leave existence open.

Declarations

Statement of AI Use. The author used large language models for code maintenance, test generation, and LaTeX editing. Kernel checks, regression tests, and retained solver logs cover only the statements and files identified in the text. Model output is not an independent mathematical review.

Conflict of Interest. The author declares no competing financial or non-financial interests.

Data and Code Availability. The Lean 4 library, the Python SAT compilers, the regression tests, and the execution scripts are available at `https://github.com/antoniomachuca/conway-99-graph`. The checked SAT pair is the CNF `p cnf 177097 422508` and the 26,746,488,053-byte proof, together with the CaDiCaL 1.9.5 log and the `drat-trim` log, in the Zenodo record 10.5281/zenodo.22995101 [1]. The earlier record 10.5281/zenodo.22983776 stores this CNF beside a 192,218,491-byte proof. Zenodo record 10.5281/zenodo.22995101 holds the canonical order-7 CNF and its proof. The exterior-row CNF is a different formula. Its checker log contains `s VERIFIED`, and neither that CNF nor its proof is in the Zenodo record. The repository files `tests/production_positive_control_results.json` and `tests/z7_general_run_result.json` record the positive-control output and the bounded quotient-level run, with SHA-256 digests. The partial proof file of that run is not deposited, because it is not a certificate. `tests/z7_general_run_result.json` records the 397,296-variable run, not the exterior-row CNF.

Acknowledgments. The author thanks the developers of CaDiCaL, `drat-trim`, PySAT, and Lean 4 for their open-source tools.

References

[1] Antonio Machuca. Order-7 CNF refutation for a Conway 99-graph encoding (CaDiCaL 1.9.5), 2026. URL https://doi.org/10.5281/zenodo.22995101. Version v2. The earlier record 10.5281/zenodo.22983776 stores a 192218491-byte proof and is not this certificate.

[2] Andries E. Brouwer and Willem H. Haemers. *Spectra of Graphs*. Universitext. Springer, 2012. URL https://homepages.cwi.nl/~aeb/math/ipm/.

[3] Patrick G. Cesarz and Andrew J. Woldar. On the automorphism group of a putative Conway 99-graph. *Algebraic Combinatorics*, 8(2):379–398, 2025. doi: 10.5802/alco.418.

[4] John H. Conway. Five $1,000 problems. Circulated prize list, *Five $1,000 Problems (Update 2017)*, Problem 2, 2017. Cheswick update dated 13 October 2014; Cheswick–Sloane postscript dated 8 June 2017.

[5] A. A. Makhnev and I. M. Minakova. On automorphisms of strongly regular graphs with the parameters $\lambda = 1$ and $\mu = 2$. *Discrete Mathematics and Applications*, 14(2):201–210, 2004. doi: 10.1515/156939204872374.

[6] A. A. Makhnev. Symmetric graphs and their automorphisms. Lecture slides, Berlin, September 2009, 2009. URL https://www.math.uni-bielefeld.de/~baumeist/sommerschule/makhnev.pdf. Theorem 1 attributes the prime-order fixed-point classification to Makhnev and Minakova.

[7] Majid Behbahani and Clement W. H. Lam. Strongly regular graphs with non-trivial automorphisms. *Discrete Mathematics*, 311(2–3):132–144, 2011. doi: 10.1016/j.disc.2010.10.005.

[8] Dean Crnković and Marija Maksimović. Construction of strongly regular graphs having an automorphism group of composite order. *Contributions to Discrete Mathematics*, 15(1):22–41, 2020. doi: 10.55016/ojs/cdm.v15i1.62323.

[9] Aalok Thakkar and Simone Severini. A forced-structure reduction and verifiable bounds for Conway's 99-graph. *arXiv preprint arXiv:2608.11211*, 2026.

[10] James M. Crawford, Matthew L. Ginsberg, Eugene M. Luks, and Amitabha Roy. Symmetry-breaking predicates for search problems. In *Proceedings of the 5th International Conference on Principles of Knowledge Representation and Reasoning (KR'96)*, pages 148–159. Morgan Kaufmann, 1996.

[11] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021.

[12] Elwyn R. Berlekamp, Jacobus H. van Lint, and Johan J. Seidel. A strongly regular graph derived from the perfect ternary Golay code. In *A Survey of Combinatorial Theory*, pages 25–30. North-Holland, Amsterdam, 1973. doi: 10.1016/B978-0-7204-2262-7.50008-9.

[13] Soe Soe Zaw. On strictly Deza graphs derived from the Berlekamp–van Lint–Seidel graph. arXiv preprint arXiv:1907.02800v2, 2020.

[14] Armin Biere. CaDiCaL at the SAT race 2019. In *SAT Race 2019*, volume B-2019-1 of *Department of Computer Science Series of Publications B*, pages 8–9. University of Helsinki, 2019.

[15] Nathan Wetzler, Marijn J. H. Heule, and Warren A. Hunt, Jr. DRAT-trim: Efficient checking and trimming using expressive clausal proofs. In *Theory and Applications of Satisfiability Testing – SAT 2014*, Lecture Notes in Computer Science. Springer, 2014. doi: 10.1007/978-3-319-09284-3_31.